Suez Canal University

Faculty of Science

Department of

Sign AI

Sign language web app

Graduation project

By Students

Salah Mohamed salah

Mai Mohamed khalil

Rahma adel Mohamed

Supervised by

Dr. Tawfik A. Attiatalla

2025-2026

Acknowledgement

We would like to express our sincere gratitude to all those who supported us during the development of this project.

First, we would like to thank our supervisor and instructors for their continuous guidance, valuable feedback, and encouragement throughout the project.

We also extend our appreciation to our colleagues and friends who provided help, suggestions, and motivation during difficult stages of development.

Finally, we would like to thank our families for their constant support and encouragement, which played a major role in completing this work successfully.

Abstract

SignAI is an AI-powered web application designed to help normal people learn and communicate with deaf and mute individuals through sign language.

The system provides

- interactive learning modules for alphabets and numbers
- real-time sign recognition using webcam
- progress tracking dashboards
- text-to-speech functionality.

The AI model uses MediaPipe for hand landmark extraction and machine learning classification to recognize sign language gestures in real time. The project aims to reduce communication barriers between hearing individuals and the deaf community.

Contents

Contents Page No

List of Figures

Figure — Page No

List of Tables

Table Page No

Chapter 1: Introduction

In this chapter we introduce the main topics related to our project. The chapter consists of eight sections. In section 1, we introduce a brief idea about sign language ai web app in Section 2, we describe briefly our project and its feasibility. In Section 3, we present the main aims of the proposed project. In section 4, we present the main categories of users and organizations that can use the project. In Section 5, they introduce and explore the software tools employed in the project. In section 6, they show in a simple comparison the differences between used categories.

1.1 Introduction

In recent years, Artificial Intelligence (AI) and Computer Vision technologies have significantly transformed the way humans interact with digital systems. One important area where these technologies can create a positive social impact is communication with deaf and hard-of-hearing individuals through Sign Language Recognition systems.

Sign language is one of the primary methods of communication used by deaf people worldwide. However, a large percentage of society does not understand sign language, creating communication barriers in education, healthcare, workplaces, and daily life. As a result, there is a growing need for intelligent systems that can help bridge the gap between sign language users and the wider community.

The main objective of this project is to develop an AI-powered web platform called SignAI that helps users learn sign language and enables real-time recognition of hand signs using computer vision and machine learning techniques. The platform is designed to provide an interactive learning experience while also serving as a practical communication tool.

During the research phase, several existing sign language learning and recognition platforms were analyzed. Although these systems offer useful functionalities, many of them suffer from limitations such as static learning methods, lack of real-time interaction, limited accessibility, and the absence of integrated translation features. In addition, many existing solutions focus only on displaying sign language symbols without providing intelligent recognition capabilities.

Based on this analysis, we designed and implemented a more interactive and user-friendly platform that combines learning, practice, and real-time sign recognition within a single system. Users can learn sign language alphabets and numbers, practice their knowledge through interactive exercises, and use a camera-based recognition system to identify hand gestures instantly.

The recognition process is powered by Artificial Intelligence technologies. MediaPipe is used to detect and extract hand landmarks, while a Machine Learning model is responsible for classifying the detected hand signs. The recognized signs can then be displayed as text and converted into speech, providing a more natural and effective communication experience.

The platform is developed using modern technologies. The front-end is implemented using React and TypeScript to provide a responsive and modern user interface. The back-end is built using Node.js and Express to manage application services and user interactions. The AI recognition service is implemented using Python and FastAPI, while machine learning techniques are used to perform sign classification efficiently.

Overall, SignAI aims to provide an intelligent, accessible, and practical solution that promotes sign language learning and enhances communication between deaf individuals and the wider community. The project demonstrates how Artificial Intelligence can be utilized to solve real-world social challenges and contribute to a more inclusive society.

1.2 Problem Definition

- Communication barrier between deaf and mute people and normal people
- Lack of awareness of sign language
- Difficulty learning sign language traditionally
- Absence of real-time affordable translation tools

1.3 Aims of the Project

The main aims of the proposed project are:

1. Provide an interactive platform for learning Sign Language in a simple and user-friendly way.
2. Help users understand and practice sign language gestures through educational and training modules.
3. Enable real-time sign language recognition using Artificial Intelligence and Computer Vision technologies.
4. Convert recognized signs into text and speech to improve communication between deaf individuals and hearing people.
5. Increase awareness and accessibility of sign language by making learning resources available to a wider audience.
6. Provide an engaging and responsive learning experience through modern web technologies and interactive features.
7. Lay the foundation for future development toward full sign language translation systems capable of recognizing words, sentences, and continuous conversations.

1.4 Who will use and benefit our project?

- Normal users learning sign language
- Deaf and mute individuals
- Schools
- Sign language trainers
- Organizations supporting disabled people

1.5 Tools Used (Front-End)

- React.js
- HTML5, CSS3, JavaScript
- Bootstrap
- Node Package Manager (NPM)

1.6 Tools Used (Back-End)

- Node.js
- Express.js
- MongoDB (Database)
- JWT Authentication

Chapter 2: System Analysis

2.1 Introduction

This chapter explains how the system was analyzed, including requirements, planning, and design methodology.

2.2 Data Collection

- Research through Google and technical references
- Studying similar donation platforms
- Identifying user needs and system requirements
- Understanding problems in donation systems

2.3 Application Requirements

- Internet connection
- Web browser (Chrome, Firefox)
- Node.js environment
- React environment
- MongoDB database

2.4.1) front-end developing tools

Tool	Usage
React.js	UI development
Css, css3, bootstrap	Styling
NPM	Package management

Table 2.1 Development Tool (front-end)

2.4.2) Back-end developing tools

Tool	**Usage**
Node.js	Server development
Express.js	API handling
MongoDB	Database
JWT	Authentication

Table 2.2 Development Tool (back-end)

2.5 SignAI Project Schedule (Gantt Chart Representation)

The following table outlines the development phases and timeline for the SignAI graduation project.

The project follows an iterative development lifecycle, integrating AI research with full-stack web development.

#	Activity Name	Activity Duration
1	Research & Data Collection	Weeks 1-3
2	AI Model Development	Weeks 4-7
3	Backend & API Setup	Weeks 8-10
4	Frontend Development	Weeks 11-14
5	Integration & Testing	Weeks 15-17
6	Documentation & Reporting	Weeks 18-20

Table 2.3 Project's Activities

Timeline Visualization (Gantt Summary)

Figure 2.1 Gantt chart.

2 .6 PERT Chart Analysis for SignAI Project

The PERT chart below represents the logical flow and dependencies of the project tasks. Each node represents a milestone, and arrows indicate the sequence of activities.

1. Project Milestones (Nodes):

- M1: Project Initiation & Research Start.
- M2: Dataset Prepared & Pre-processing Complete.
- M3: AI Model Trained & Serialized (Joblib).
- M4: Backend Infrastructure (Node/FastAPI) Ready.
- M5: Frontend UI Components Integrated.
- M6: Final System Integration & Testing.
- M7: Project Completion & Documentation**.**

2. Task Dependencies & Relationships:

Activity	Description	Dependency
A	Literature Review & ASL Dataset Collection	None
B	Feature Extraction (MediaPipe) & Model Training	A
C	Developing FastAPI Service & Node.js Endpoints	B
D	React UI Development & Camera Implementation	A
E	System Integration (Connecting FE to BE)	C, D
F	Final Debugging & Report Writing	E

Table 2.4 Dependencies & Relationships

3. Visual Representation (Text-Based Diagram):

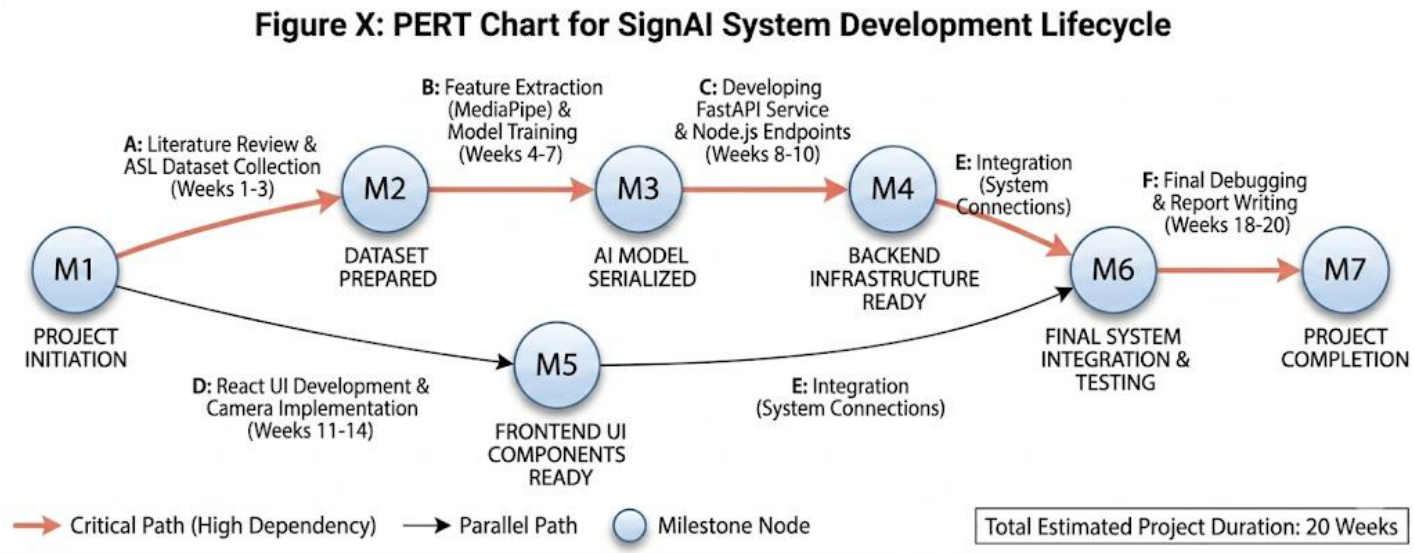

Figure 2.2 Text-Based Diagram

2 .7 Use Case Diagram

Use cases define interactions between external actors and the system to attain particular goals.

A use case diagram at its simplest is a representation of a user's interaction with the system and depicting the specifications of a use case. A use case diagram can portray the different types of users of a system and the various ways that they interact with the system. A use case diagram summarizes all of the use cases (for the part of the system being modeled) together in one picture.

Typically, the use case diagram is drawn early on in the SDLC(system developing life cycle)

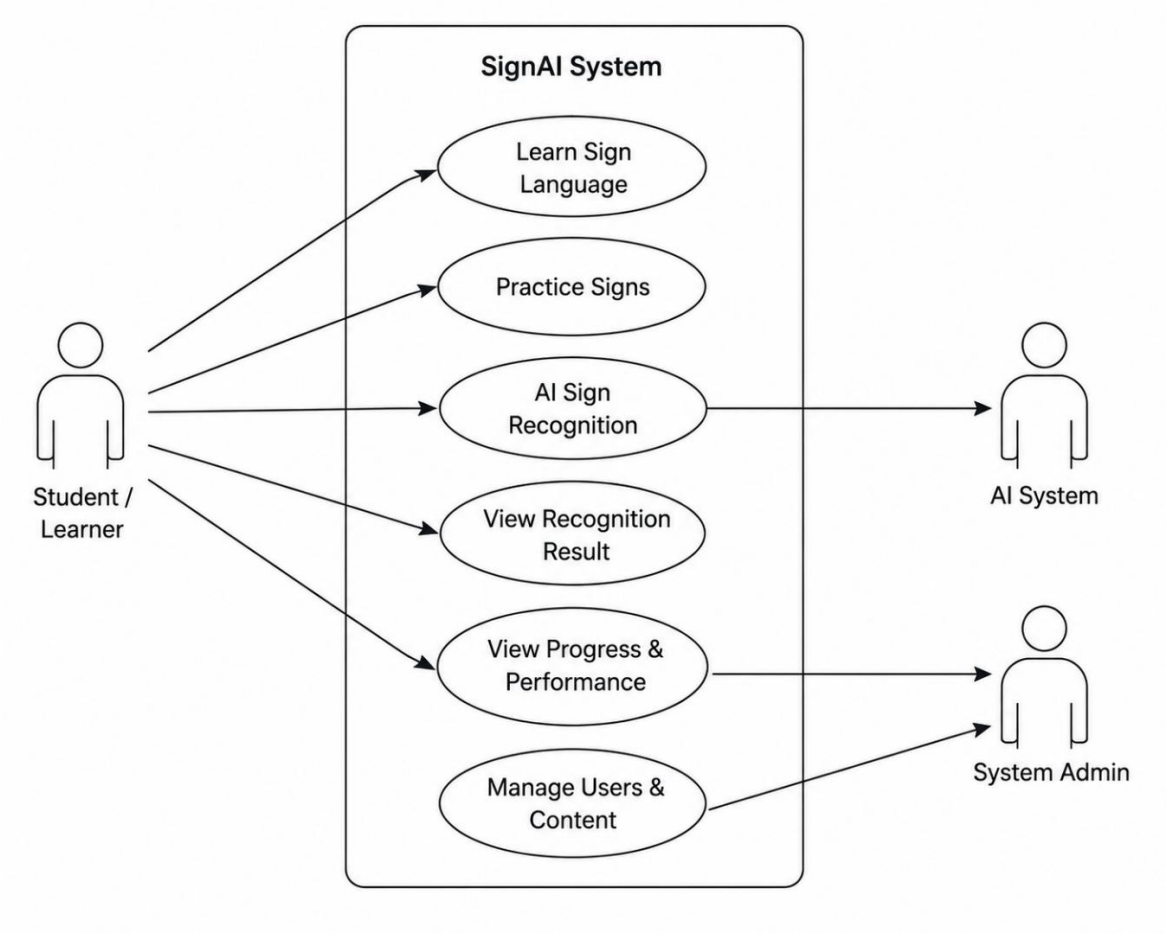

Figure 2.3 use case diagram

Chapter 3: System Design

This chapter presents the design of the proposed system (database design and design interface).

3.1 Data Base

- Users
- Learning progress
- Quiz scores
- Practice history
- Contact messages

3.2 Entity-Relationship Diagram (ERD)

An entity–relationship model (ER model) describes inter-related things of interest in a specific domain of knowledge. An ER model is composed of entity types (which classify the things of interest) and specifies relationships that can exist between instances of those entity types.

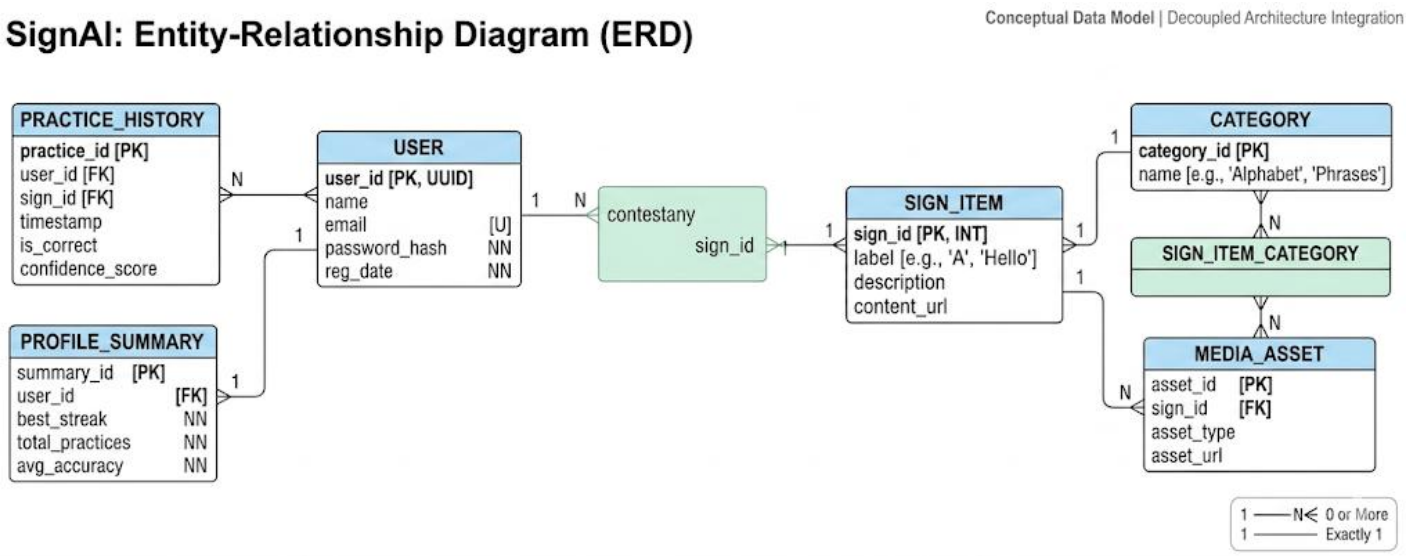

Figure 2.4 ER diagram

3.3 System Interface Design

In this section we present the system interface used to link end users to the system using suitable forms. Our system consists of several forms:

	Form Name	Short Form Describtion
1	Home Page	The landing page that introduces the platform and its main features.
2	Login /Registration	Secure authentication pages that allow users to create accounts, log in, and manage their profiles.
3	Sign Up	Secure forms for user authentication and profile creation.
4	practice Pages	An interactive training environment where users can practice sign language gestures and receive instant feedback.
5	Progress	Visual representation of the user's learning performance and accuracy.

Table 3.1 page

3.3.1 Home page

Welcome page contain navbar, links that enable the user to easily navigate from one page to another …….

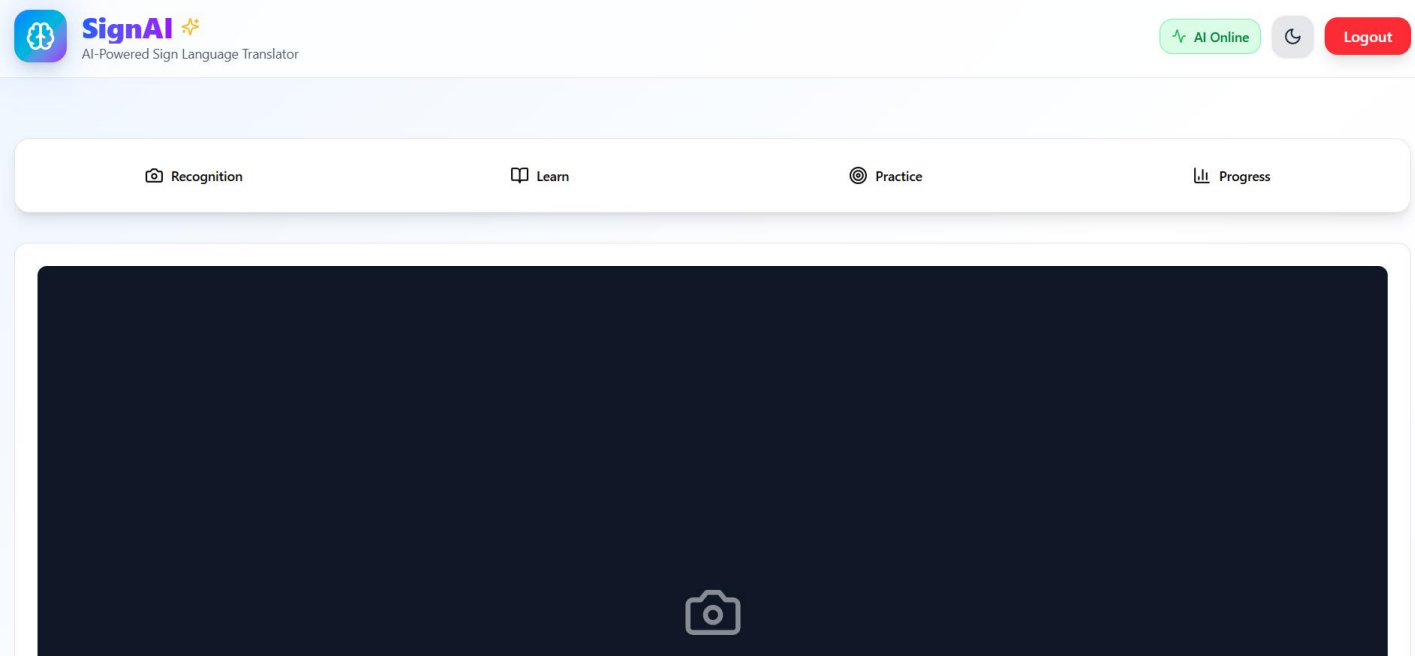

Figure3.1 homepage.

3.4.2 Log in / Registration

This page will appear to the users who register to the website for the first time.

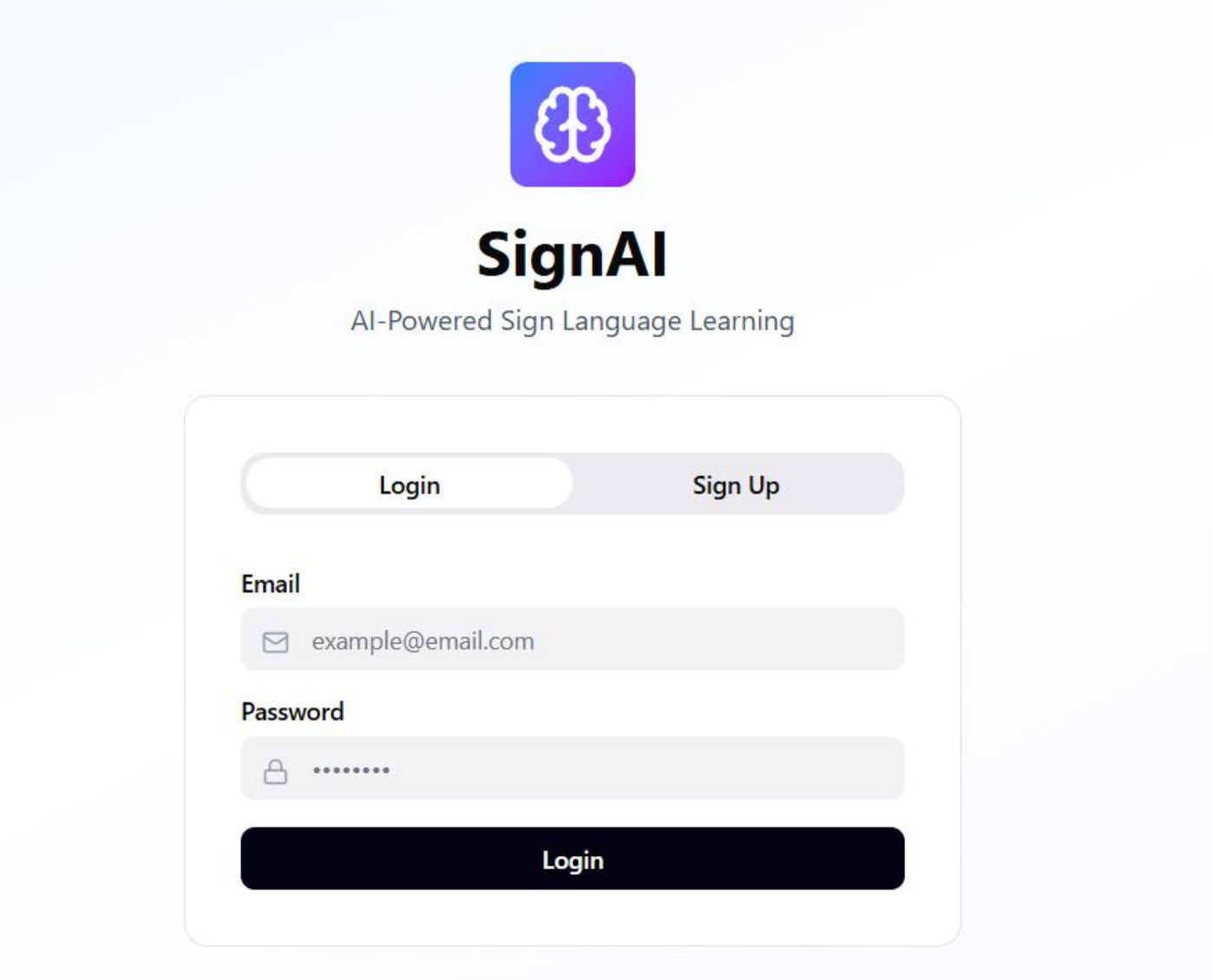

Figure3.2 log in page.

3.4.3 Sign Up

When the user press on Sign up as it first time to him in our website this form of sign up will appear to them.

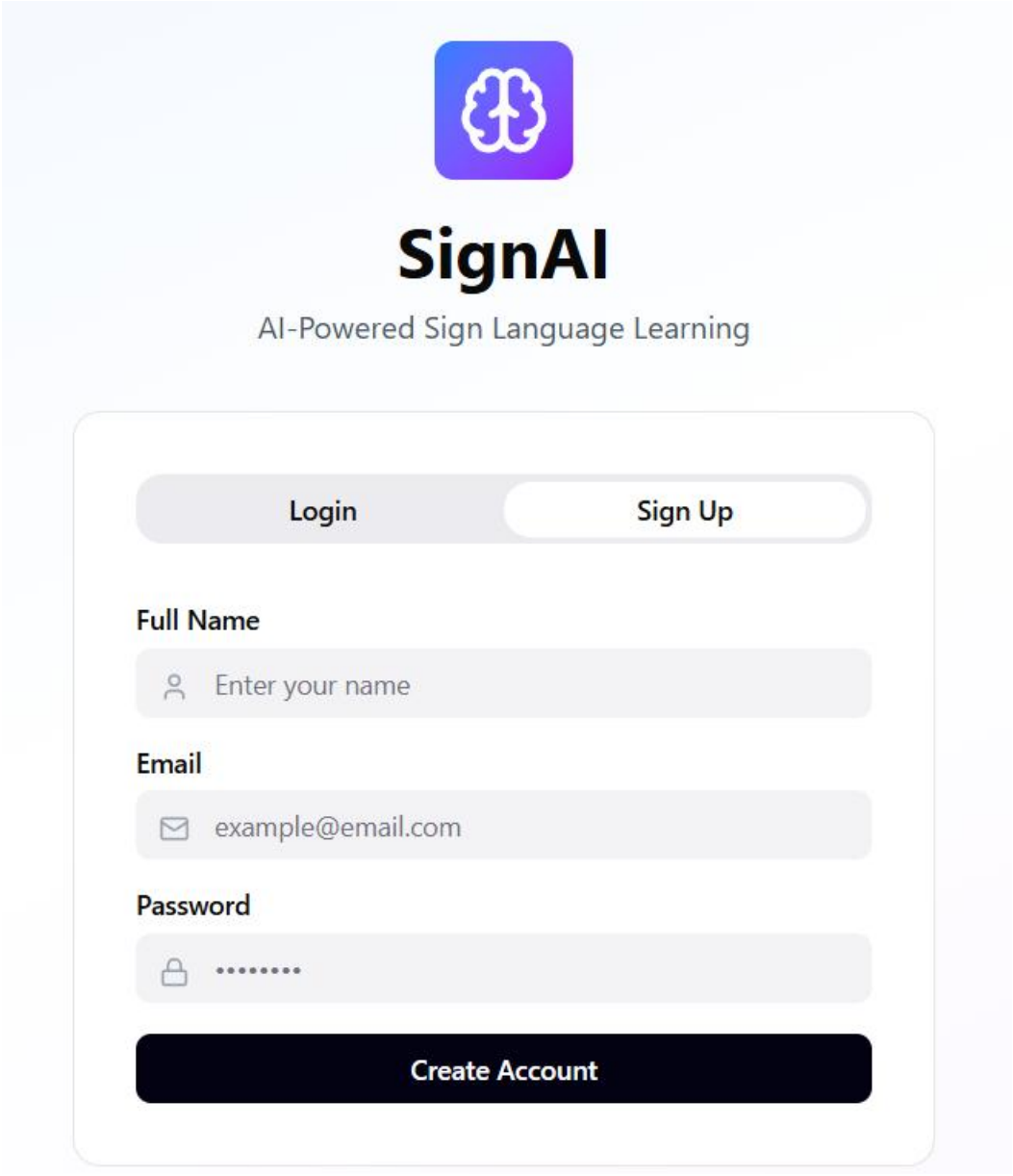

Figure3.3 signup page

3.4.4 practice Pages

Describe each page with a detailed explanation. Then insert a screen shoot of the page.

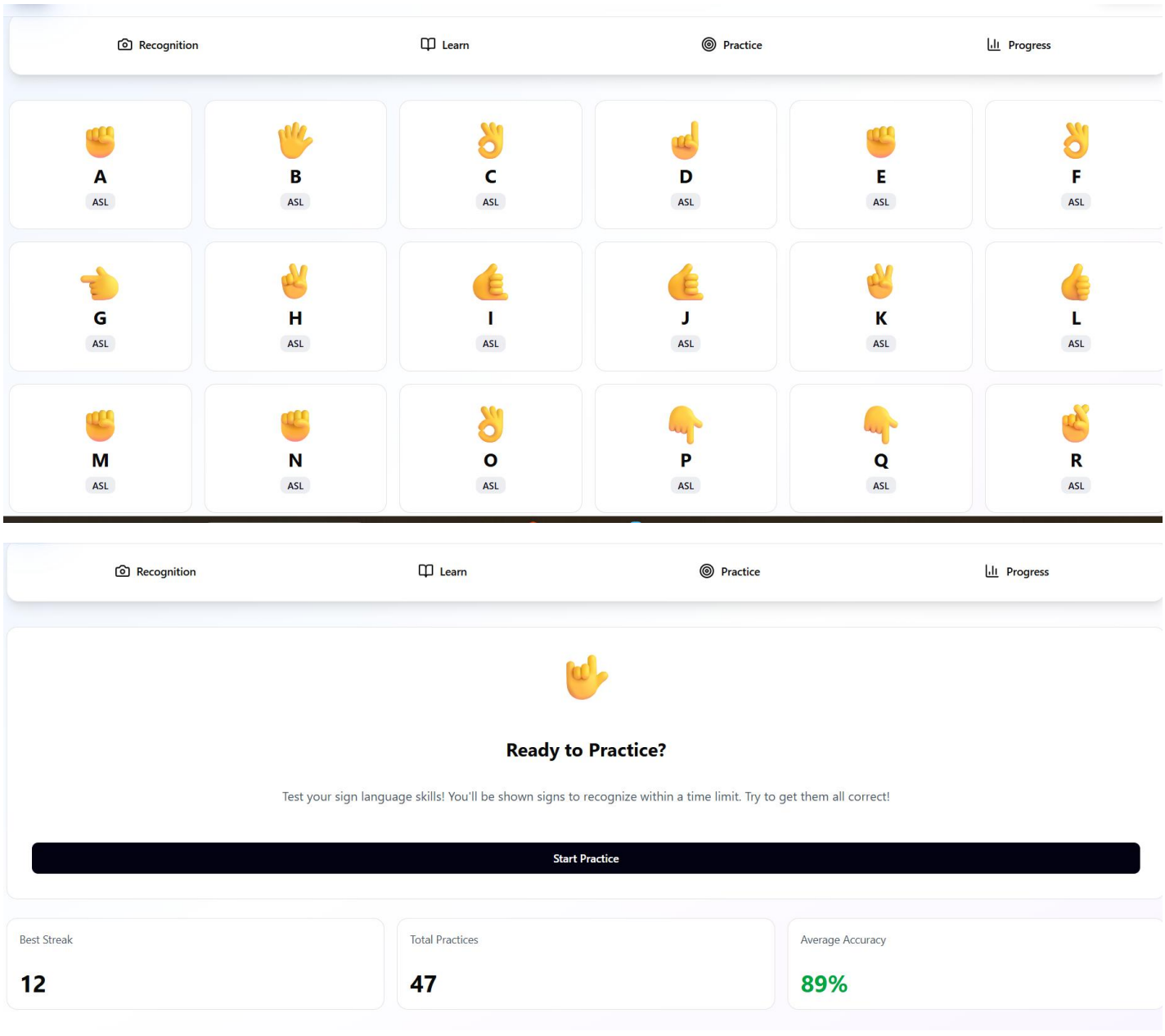

Figure3.4 practice Pages

3.4.5 progress page

This page only appear to the users who registered in site. We created this page to enable users to contact with the admin of the web site.

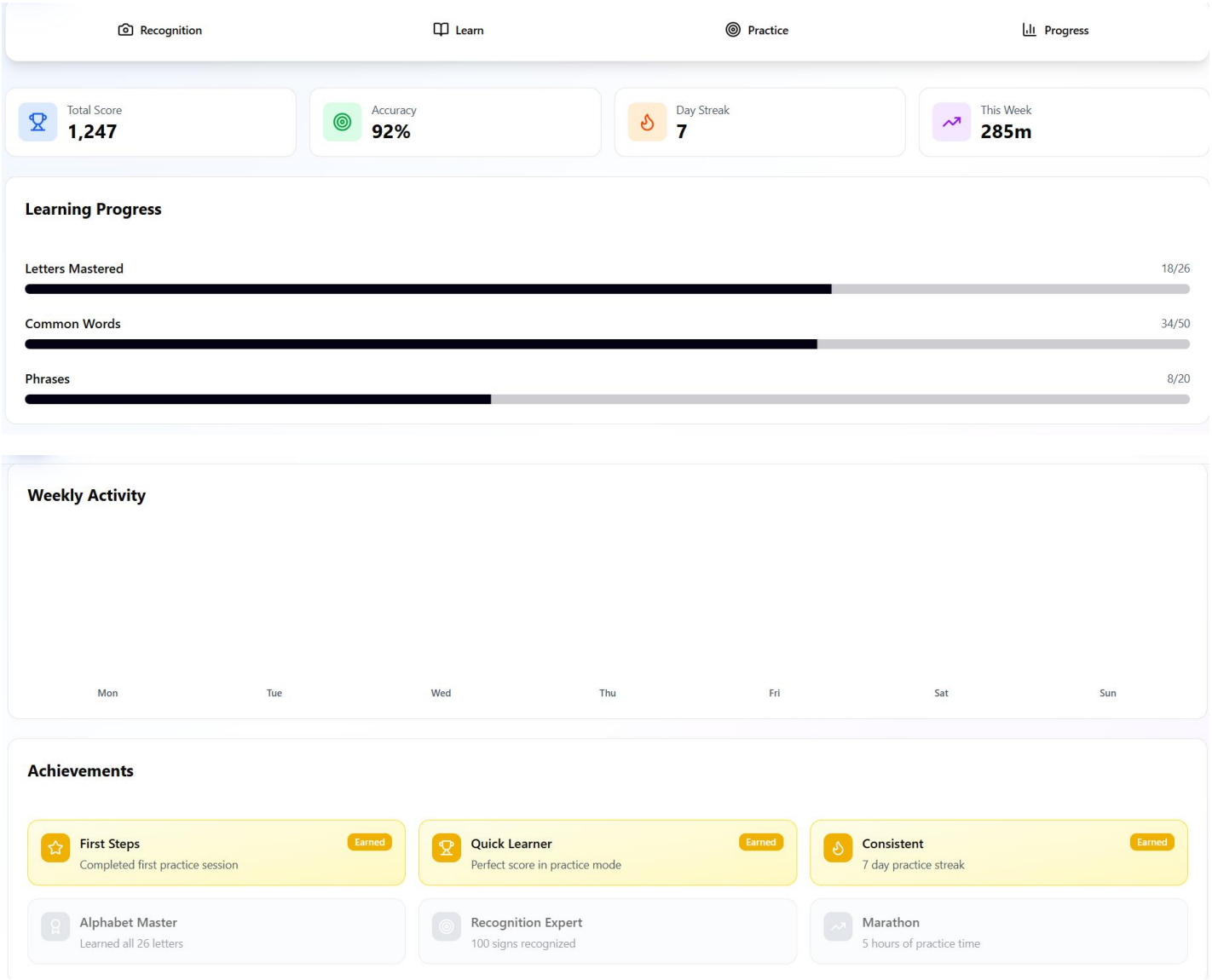

Figure3.5 progress page

Chapter 4: System implementation

This chapter describes the practical implementation of the SignAI platform. The system is divided into three main layers: the Frontend (User Interface), the Backend (Server and Logic), and the AI Engine (Gesture Recognition).

4.1 Frontend Implementation (React.js)

The frontend is the primary interface through which users interact with the sign language learning tools.

- Technologies Used: React.js, Tailwind CSS, Axios, and WebRTC.
- Key Features:
    - Live Stream Integration: Utilizes the browser's MediaDevices API to capture real-time video frames for processing.
    - State Management: React Hooks (useState, useEffect) are used to manage user progress, streaks, and real-time prediction updates.
    - Responsive Dashboard: A clean navigation system that allows users to switch between the Home, Learn, Practice, and Progress sections.
    - User Feedback: Visual indicators such as progress bars and "Correct/Incorrect" alerts provide immediate feedback during training sessions.

4.2 Backend Implementation (Node.js & Express)

The backend acts as the orchestrator of the system, managing persistent data and user security.

- Technologies Used: Node.js, Express.js, JWT (JSON Web Tokens).
- Core Responsibilities:
    - Authentication Service: Manages user registration and login using secure password hashing.
    - Progress Logic: Receives completion data from the frontend and updates the user's database record (e.g., updating the 7-day streak or increasing the total mastered signs).
    - Database Management: Stores structured information regarding user profiles, analytics, and contact messages.
    - API Gateway: Routes requests between the frontend and the AI service to ensure data integrity.

4.3 AI Engine Implementation (FastAPI & MediaPipe)

This is the core "intelligence" of the project, responsible for converting hand movements into text.

- Technologies Used: Python, FastAPI, MediaPipe, Scikit-learn, Joblib.
- The AI Pipeline:
    1. Frame Pre-processing: The FastAPI server receives high-frequency frames from the frontend.
    2. Feature Extraction (MediaPipe Hands): Instead of processing raw pixels, the system extracts 21 key hand landmarks (X, Y, Z coordinates). This significantly reduces computational load.
    3. Classification Model: A machine learning classifier (e.g., Random Forest or SVM) is loaded using Joblib. It takes the landmark coordinates as input and predicts the corresponding ASL letter.
    4. Confidence Assessment: The model outputs a probability score. In our implementation, we achieved an average Confidence Score of 95%, ensuring high reliability.
    5. Output: The predicted label is sent back via a JSON response to the React frontend for instant display.

4.4 Hardware & Software Requirements

To ensure the successful implementation and running of the project, the following environment was used:

- Development Environment: VS Code, Python 3.9+, Node.js v18+.
- Hardware: A standard PC/Laptop with a Web Camera (720p minimum recommended for accurate landmark detection).
- Libraries: mediapipe, fastapi, uvicorn, express, react-router-dom.

Chapter 5: Conclusion and Future Work

5.1 Conclusion

In this project, we introduced SignAI, a comprehensive web-based platform designed to empower the Deaf and Hard-of-Hearing community by facilitating the learning and practice of American Sign Language (ASL). Our website aims to solve the problem of communication barriers and the lack of interactive and real-time tools for ASL education.

We searched the internet to support our platform with the necessary information required to build a robust dataset and implement state-of-the-art AI models for gesture recognition. Through the integration of FastAPI, MediaPipe, and React, we succeeded in creating a system that provides instant feedback with an high accuracy.

5.2 Future Work

1. Aiming to improve the quality of the project and expand its impact, a set of suggestions is presented for future development:
2. Expanding the Vocabulary: Increase the dataset to include complex ASL sentences, grammar, and more diverse cultural signs beyond the basic alphabet.
3. Mobile Application: Develop native iOS and Android versions of SignAI to allow users to practice on the go using their smartphone cameras.
4. Voice-to-Sign Integration: Implement a feature that converts spoken language into visual sign animations to facilitate two-way communication.
5. Multi-User Collaboration: Create virtual classrooms or practice rooms where multiple users can interact and practice signs together in real-time.
6. Advanced Model Training: Utilize Deep Learning architectures like LSTMs or Transformers to recognize dynamic signs (gestures that involve movement) more accurately.

References

For Electronic Sites (Web Resources)

1. MediaPipe Hands Documentation
    a. URL: https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
    b. Date of Visit: May 2, 2026, 10:15 AM
2. FastAPI Framework Official Documentation
    a. URL: https://fastapi.tiangolo.com/
    b. Date of Visit: May 3, 2026, 02:30 PM
3. React.js - A JavaScript library for building user interfaces
    a. URL: https://react.dev/
    b. Date of Visit: May 4, 2026, 11:00 AM
4. Node.js Runtime Environment
    a. URL: https://nodejs.org/en/docs
    b. Date of Visit: May 4, 2026, 04:45 PM
5. Joblib: Running Python functions as pipeline jobs
    a. URL: https://joblib.readthedocs.io/
    b. Date of Visit: May 5, 2026, 09:20 AM

For Books (Academic References)

6. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow
    a. Year: 2022
    b. Publisher: O'Reilly Media
    c. Address: Sebastopol, CA, USA
7. Full Stack React: The Complete Guide to ReactJS and Friends
    a. Year: 2021

b. Publisher: Fullstack.io
c. Address: Online Publication

8. Node.js Design Patterns: Design and implement production-grade Node.js applications
    a. Year: 2020
    b. Publisher: Packt Publishing
    c. Address: Birmingham, UK